

## ASSIGNMENT NO 5

### Aim

To implement the **N-Queen problem using the Constraint Satisfaction Problem (CSP)** approach and place **N queens on an  $N \times N$  chessboard** such that no two queens attack each other.

### Objectives

1. To understand the concept of a **Constraint Satisfaction Problem (CSP)**.
2. To formulate the N-Queen problem using **variables, domains, and constraints**.
3. To apply **backtracking search** for solving the N-Queen problem.
4. To ensure that no two queens are placed in the same **row, column, or diagonal**.
5. To find one or more valid solutions for a given value of N.

### Theory

A **Constraint Satisfaction Problem (CSP)** is a problem defined using:

- **Variables** – Objects whose values need to be determined.
- **Domains** – Possible values that can be assigned to each variable.
- **Constraints** – Rules that restrict the possible assignments.

### N-Queen Problem

The N-Queen problem requires placing **N queens on an  $N \times N$  chessboard** so that no two queens attack each other.

A queen can attack another queen if they are in the same:

- Row
- Column
- Diagonal

### Backtracking Approach

The problem can be solved using **backtracking**. Queens are placed row by row. Before placing a queen, the algorithm checks whether the position satisfies all constraints.

If a valid position cannot be found for the current row, the algorithm **backtracks** to the previous row and changes the position of the previously placed queen.

### Algorithm

#### Algorithm: N-Queen using CSP and Backtracking

1. Start with an empty  $N \times N$  chessboard.
2. Start placing queens from the first row.
3. For the current row, try every column from 1 to N.

## IV

4. Check whether placing a queen at (row, column) is safe.
5. A position is safe if:
  - No queen exists in the same column.
  - No queen exists on the upper-left diagonal.
  - No queen exists on the upper-right diagonal.
6. If the position is safe: Place the queen. Move to the next row.
7. If all  $N$  queens are successfully placed, display the solution.
8. If no safe position is available: Remove the queen from the previous row. Try another position.
9. Repeat the process until a solution is found or all possibilities are exhausted.
10. Stop.

Python code :

```
# Python program to solve N Queen
# Problem using backtracking

global N
N = 4

def printSolution(board):
    for i in range(N):
        for j in range(N):
            print (board[i][j],end=' ')
        print()

# A utility function to check if a queen can
# be placed on board[row][col]. Note that this
# function is called when "col" queens are
# already placed in columns from 0 to col -1.
# So we need to check only left side for
# attacking queens
def isSafe(board, row, col):

    # Check this row on left side
    for i in range(col):
        if board[row][i] == 1:
            return False
```

```

# Check upper diagonal on left side
for i, j in zip(range(row, -1, -1), range(col, -1, -1)):
    if board[i][j] == 1:
        return False

# Check lower diagonal on left side
for i, j in zip(range(row, N, 1), range(col, -1, -1)):
    if board[i][j] == 1:
        return False

return True

def solveNQUtil(board, col):
    # base case: If all queens are placed
    # then return true
    if col >= N:
        return True

    # Consider this column and try placing
    # this queen in all rows one by one
    for i in range(N):

        if isSafe(board, i, col):
            # Place this queen in board[i][col]
            board[i][col] = 1

            # recur to place rest of the queens
            if solveNQUtil(board, col + 1) == True:
                return True

            # If placing queen in board[i][col]
            # doesn't lead to a solution, then
            # queen from board[i][col]
            board[i][col] = 0

    # if the queen can not be placed in any row in
    # this column col then return false
    return False

# This function solves the N Queen problem using
# Backtracking. It mainly uses solveNQUtil() to
# solve the problem. It returns false if queens
# cannot be placed, otherwise return true and
# placement of queens in the form of 1s.
# note that there may be more than one
# solutions, this function prints one of the
# feasible solutions.
def solveNQ():
    board = [ [0, 0, 0, 0],
               [0, 0, 0, 0],
               [0, 0, 0, 0],
               [0, 0, 0, 0]
             ]

```

```
if solveNQUtil(board, 0) == False:
    print ("Solution does not exist")
    return False

printSolution(board)
return True

# driver program to test above function
solveNQ()

# This code is contributed by Divyanshu Mehta
```

---

### Output:

```
0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0
```

## Conclusion

The N-Queen problem was successfully formulated and solved as a **Constraint Satisfaction Problem** using backtracking. The solution demonstrates how variables, domains, and constraints can be used to systematically search for a valid assignment. The algorithm ensures that no two queens are placed in the same row, column, or diagonal and effectively uses **backtracking to eliminate invalid configurations**.